

Projeto e Análise de Algoritmos I

Aula 02 - Emparelhamento Estável

Lucas Nunes Alegre

Universidade Federal do Rio Grande do Sul
Instituto de Informática
Departamento de Informática Teórica



Estes slides utilizam conteúdo adaptado da bibliografia da disciplina e também de notas de aula e slides prévios dos professores Bruno Grisci, Rodrigo Machado e André Grahl.

Emparelhamento Estável

Emparelhamento entre Hospitais e Médicos

Objetivo: Dado um conjunto de preferências entre hospitais e estudantes de medicina, definir um processo de admissão **autoimposto**.



Par Instável: estudante e e hospital h são instáveis se:

- e prefere h ao seu hospital atribuído; e
- h prefere e a um dos seus estudantes aprovados.

Emparelhamento Estável:

- Interesse próprio força a atribuição e previne acordos unilaterais.

Problema de Emparelhamento Estável

Objetivo: Dado um conjunto de n hospitais e um conjunto de n estudantes, encontrar uma atribuição adequada.

- Participantes ranqueiam membros do grupo oposto.
- Cada hospital lista estudantes em ordem de preferência da melhor para a pior.
- Cada estudante lista hospitais em ordem de preferência do melhor para o pior.

Exemplo de Instância (3 hospitais, 3 estudantes):

Preferências dos Hospitais:

- $h_1: e_1 \succ e_2 \succ e_3$
- $h_2: e_2 \succ e_1 \succ e_3$
- $h_3: e_1 \succ e_2 \succ e_3$

Preferências dos Estudantes:

- $e_1: h_2 \succ h_1 \succ h_3$
- $e_2: h_1 \succ h_2 \succ h_3$
- $e_3: h_1 \succ h_2 \succ h_3$

Emparelhamento Perfeito

Def. Um **emparelhamento** S é um conjunto ordenado de pares h - e com $h \in H$ e $e \in E$ tal que cada hospital $h \in H$ e cada estudante $e \in E$ aparece no máximo em um par de S .

Def. Um emparelhamento S é **perfeito** se $|S| = |H| = |E| = n$.

Exemplo de Emparelhamento Perfeito:

- $H = \{h_1, h_2, h_3\}$ e $E = \{e_1, e_2, e_3\}$
- $S = \{(h_1, e_2), (h_2, e_1), (h_3, e_3)\}$

Par Instável

Def. Dado um emparelhamento perfeito S , um hospital h e uma estudante e são um **par instável** se:

- h prefere e à sua opção atual; e
- e prefere h à sua opção atual.

Obs. Um par instável (h, e) pode melhorar suas opções com uma ação conjunta.

Exemplo:

Preferências:

Emparelhamentos:

- $\{(h_1, e_1), (h_2, e_2)\}$ é **estável**
- $\{(h_1, e_2), (h_2, e_1)\}$ é **instável**
(pois h_1 e e_1 formam um par instável)

Problema de Emparelhamento Estável

Def. Um **emparelhamento estável** é um emparelhamento perfeito sem pares instáveis.

Problema: Dadas listas de preferências de n hospitais e n estudantes, encontrar um emparelhamento estável (se existir).

- Natural, desejável, condição autoimposta.
- Interesse próprio previne qualquer hospital-estudante trocar de pares.

Problema de Emparelhamento Estável

P: Emparelhamentos estáveis sempre existem?

Problema: Colega de Quarto Estável.

- $2n$ pessoas.
- Cada pessoa ranqueia outras pessoas de 1 até $2n - 1$.
- Atribuir pares de colegas de quarto sem pares instáveis.
- Emparelhamentos estáveis nem sempre existem.
- Note que nesse caso não há a separação em dois grupos isolados.

Problema de Emparelhamento Estável

P: Emparelhamentos estáveis sempre existem?

	1 ^a	2 ^a	3 ^a
A	B	C	D
B	C	A	D
C	A	B	D
D	A	B	C

Observações sobre instabilidades:

- A-B, C-D $\Rightarrow$ B-C instável
- A-C, B-D $\Rightarrow$ A-B instável
- A-D, B-C $\Rightarrow$ A-C instável
- Não existe emparelhamento perfeito que seja estável neste exemplo.
- **Algoritmo de Irving** encontra emparelhamento estável (quando existe):
<https://www.youtube.com/watch?v=5QLxAp8mRko>

Algoritmo Gale-Shapley

Algoritmo Gale-Shapley

Algoritmo intuitivo que **garantidamente** encontra um emparelhamento estável.

GALE-SHAPLEY(H, E)

```
1:  Inicialize  $S \leftarrow \emptyset$ , e todos os hospitais e estudantes como livres
2:  while existe um hospital livre  $h$  que ainda não propôs a todo estudante do
3:    Escolha um tal hospital  $h$ 
4:    Seja  $e$  o estudante de maior preferência de  $h$  ao qual  $h$  ainda não propôs
5:    if  $e$  está livre then
6:       $S \leftarrow S \cup \{(h, e)\}$ 
7:    else
8:       $e$  está atualmente emparelhado com  $h'$ 
9:      if  $e$  prefere  $h'$  a  $h$  then
10:         $h$  continua livre
11:      else
12:         $S \leftarrow S \setminus \{(h', e)\}$ 
13:         $S \leftarrow S \cup \{(h, e)\}$ 
14:         $h'$  fica livre
15:
16:  return  $S$ 
```

Análise de Gale-Shapley

Algoritmo Gale-Shapley

O que mostrar?

Item 1: Emparelhamento perfeito.

Item 2: Sem pares instáveis.

Item 3: Algoritmo G-S termina.

Item 4: Justiça.

Prova de Corretude: Emparelhamento Perfeito

Afirmção. No Algoritmo G-S, todos os hospitais e estudantes são emparelhados.

Prova. Prova por contradição.

- Suponha para o efeito de contradição que o hospital h não está pareado quando o algoritmo termina.
- h foi rejeitado por todos os estudantes na sua lista.
- Então todos os estudantes têm um par ao fim do algoritmo que elas preferem a h .
- Mas o número de hospitais e estudantes é o mesmo. Então, todo hospital possui um par, inclusive h , o que gera uma **contradição**. ■

Algoritmo Gale-Shapley

O que mostrar?

Item 1: ~~Emparelhamento perfeito.~~

Item 2: Sem pares instáveis.

Item 3: Algoritmo G-S termina.

Item 4: Justiça.

Prova de Corretude: Emparelhamento Estável

Afirmção. No Algoritmo G-S, não existem pares instáveis.

Prova. S é perfeito. Suponha para o efeito de contradição que o par $(h, e) \notin S$ é instável.

- **Caso 1:** h nunca propôs a e .
 - h prefere seu par atual a e .
 - (h, e) é estável.
- **Caso 2:** h propôs a e .
 - e rejeitou h (imediatamente ou depois).
 - e prefere seu par atual a h .
 - (h, e) é estável.
- Em ambos os casos, o par (h, e) é estável.



Algoritmo Gale-Shapley

O que mostrar?

Item 1: ~~Emparelhamento perfeito.~~

Item 2: ~~Sem pares instáveis.~~

Item 3: Algoritmo G-S termina.

Item 4: Justiça.

Prova de Corretude: Terminação

Obs. Hospitais propõem para estudantes em ordem decrescente de preferência (**apenas piora**).

Obs. Assim que uma estudante é proposta, ela nunca fica sozinha, ela apenas troca a proposta atual por uma proposta melhor (**apenas melhora**).

P: Como provar que o algoritmo termina?

P: Como medir progresso? . . . Número de indivíduos livres? . . . Número de pares? . . . Número de propostas realizadas?

Prova de Corretude: Terminação

Afirmção. O algoritmo termina no máximo após n^2 iterações do loop.

Prova. A cada iteração do loop um hospital propõe para uma nova estudante. Existem apenas n^2 possíveis propostas. ■

Hospitais

	1 ^a	2 ^a	3 ^a	4 ^a	5 ^a
V	A	B	C	D	E
W	B	C	D	A	E
X	C	D	A	B	E
Y	D	A	B	C	E
Z	A	B	C	D	E

Estudantes

	1 ^a	2 ^a	3 ^a	4 ^a	5 ^a
A	W	X	Y	Z	V
B	X	Y	Z	V	W
C	Y	Z	V	W	X
D	Z	V	W	X	Y
E	V	W	X	Y	Z

Exemplo: Pior caso.

Pior Caso

Afirmção. O algoritmo realiza no máximo $n(n - 1) + 1$ propostas.

Prova.

- Uma estudante, ao receber uma proposta, nunca mais volta a ficar livre.
- O algoritmo termina no exato momento em que o último estudante livre é comprometido.
- No pior caso, antes dessa última proposta, os outros $(n - 1)$ estudantes já receberam propostas de todos os n hospitais.
- Isso totaliza $n(n - 1)$ propostas feitas a essas estudantes.
- A proposta seguinte é feita à estudante restante e encerra o algoritmo.
- Portanto, o número máximo de propostas é

$$\underbrace{n}_{\text{hospitais}} \underbrace{(n - 1)}_{\text{rejeições}} + \underbrace{1}_{\text{final sem rejeição}} = n^2 - n + 1.$$



Resumo

Teorema. [Gale-Shapley 1962] O algoritmo Gale-Shapley garante encontrar um emparelhamento estável para qualquer **instância** do Problema de Emparelhamento Estável.

P: Se existem múltiplos emparelhamentos estáveis, qual o algoritmo G-S irá retornar? A ordem das propostas importa?

Algoritmo Gale-Shapley

O que mostrar?

Item 1: ~~Emparelhamento perfeito.~~

Item 2: ~~Sem pares instáveis.~~

Item 3: ~~Algoritmo G-S termina.~~

Item 4: Justiça.

Justiça

Entendendo a Solução

P. A **ordem** em que os hospitais são selecionados para propor muda o resultado do algoritmo?

Def. Uma estudante e é uma **opção viável** para um hospital h se existe algum emparelhamento estável com o par (h, e) .

Exemplo:

- $h - e, e'$.
- $h' - e, e'$.
- $e - h, h'$.
- $e' - h, h'$.
- Apenas e é viável para h .

Entendendo a Solução

Def. Uma estudante e é uma **opção viável** para um hospital h se existe algum emparelhamento estável com o par (h, e) .

Atribuição Hospital-Ótima: Todo hospital recebe a melhor opção viável.

- É um emparelhamento perfeito?
- É um emparelhamento estável?

Afirmção. Toda execução do algoritmo G-S retorna uma atribuição hospital-ótima.

Corolário. Uma atribuição hospital-ótima é um emparelhamento estável.

Otimidade para Hospitais

Afirmção. O algoritmo G-S é **hospital-ótimo**: cada hospital recebe sua **melhor opção viável**.

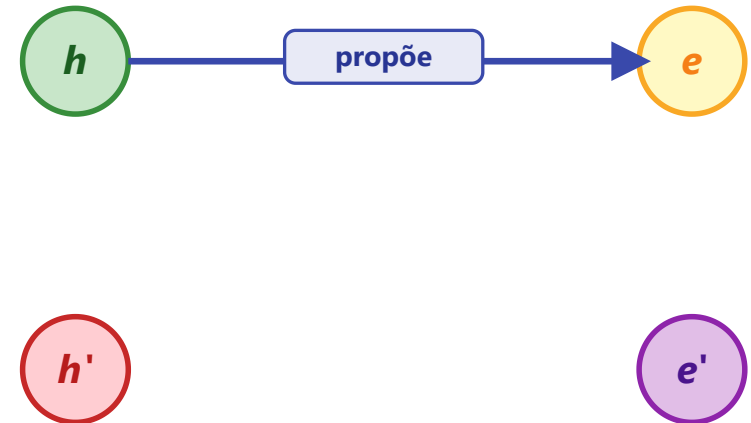
Prova. Por **contradição**.

[Hipótese] Suponha que algum hospital não termina com sua melhor opção viável.

- Como os hospitais propõem em ordem decrescente de preferência, algum hospital foi **rejeitado** por uma opção viável.
- Seja h o **primeiro** hospital rejeitado por uma opção viável, e $e = \text{best}(h)$ essa opção.
 - *(Até esse momento, nenhum outro hospital foi rejeitado por uma opção viável.)*
- Quando e rejeita h , ela se compromete com outro pretendente h' — logo e **prefere h' a h** .

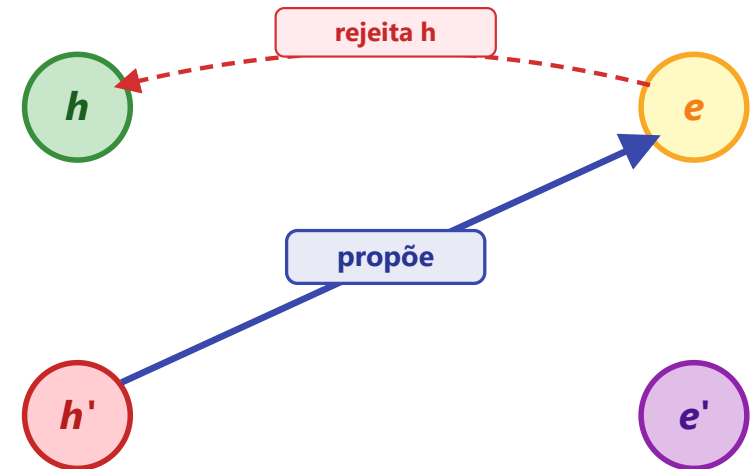
Visualizando a Contradição — Passo 1

- h propõe a e em ordem de preferência.
- e aceita a proposta.
- Forma-se o emparelhamento provisório h - e .



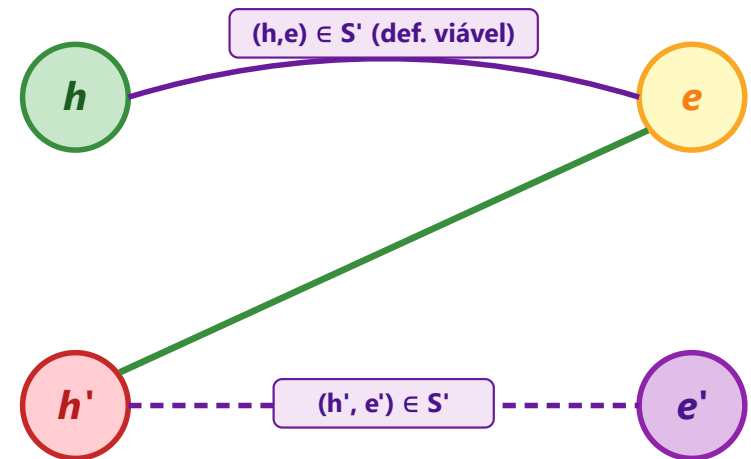
Visualizando a Contradição – Passo 2

- **1ª Rejeição:** h é o primeiro hospital rejeitado por uma opção viável (e).
- e recebe proposta de h' e **rejeita** h porque **prefere** h' a h .



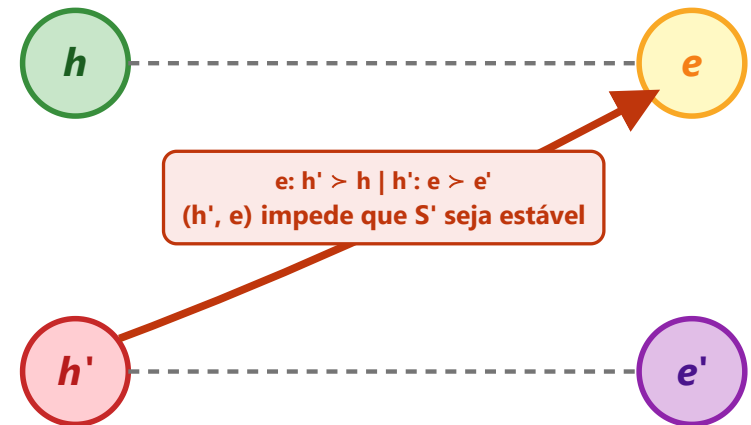
Visualizando a Contradição – Passo 3

- Como e é uma **opção viável** para h , **por definição** DEVE existir algum emparelhamento estável S' no qual o par (h, e) está junto.
- Em S' , o hospital h' precisa estar pareado com outra opção $e' \neq e$.



Visualizando a Contradição — Passo 4

- Em S' , temos (h, e) e (h', e') .
- e prefere h' a h (rejeitou h por h' na execução E).
- h' prefere e a e' (pois h foi o 1º a ser rejeitado por uma opção viável, logo h' ainda não fora rejeitado por e').



Conclusão: (h', e) é um **par instável** e S' não é estável, **contradição!** ■

Entendendo a Solução

P: O que acontece com os estudantes?

Pessimalidade para Estudantes: Cada estudante recebe a **pior** opção viável.

Pessimalidade para Estudantes

Afirmção. O algoritmo G-S é **estudante-péssimo**.

Prova. Por contradição. Suponha que o hospital h é atribuído a e em S^* , mas h não é a pior opção viável de e .

- Então existe um emparelhamento estável S' em que e é pareado com um hospital h' , de quem e gosta menos do que h .
- $\Rightarrow e$ prefere h a h' .
- Seja $e' \neq e$ a opção de h em S' . Por otimalidade para hospitais, e é a melhor opção viável para h (foram pareados em S^*).
- $\Rightarrow h$ prefere e a e' .
- Então, (h, e) é um par **instável** em S' , uma **contradição**. ■

Emparelhamento de Empresas e Alunos

Exemplo: Hospitais $\approx$ Empresas, Estudantes $\approx$ Alunos.

Variação 1: Alguns participantes declaram outros como inaceitáveis.

Variação 2: Números diferentes de hospitais e estudantes.

Variação 3: Poligamia limitada.

Def. Um emparelhamento S é **instável** se há uma empresa e e um aluno a tal que:

- e e a são aceitáveis mutuamente; e
- a não está emparelhado, ou a prefere e a sua empresa atual; e
- e não preencheu todas as suas vagas, ou e prefere a a um dos seus atuais estagiários.

Contexto Histórico

Contexto Histórico

- Nomeado em homenagem a **David Gale** e **Lloyd Shapley**, que o publicaram em 1962.
- Já era utilizado desde o início da década de 1950 no **National Resident Matching Program** (NRMP), para alocação de médicos residentes nos EUA.

Contexto Histórico

National Resident Matching Program.

- Processo de decisão centralizada.
- 38.000 estudantes e 28.000 posições.

The Redesign of the Matching Market for American Physicians: Some Engineering Aspects of Economic Design

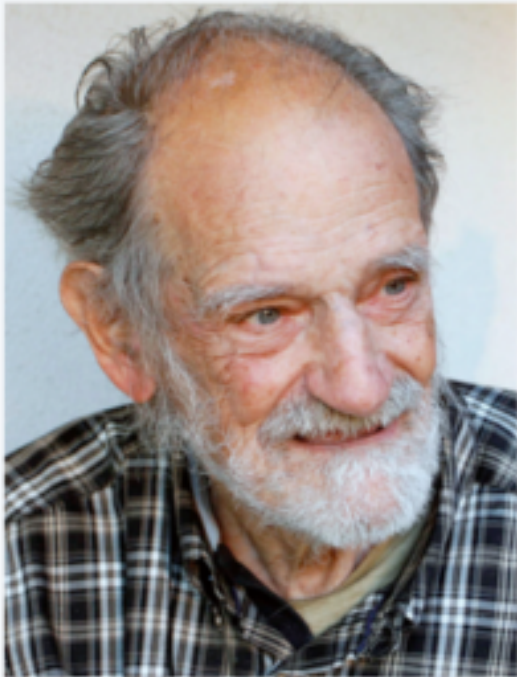
By ALVIN E. ROTH AND ELLIOTT PERANSON*

We report on the design of the new clearinghouse adopted by the National Resident Matching Program, which annually fills approximately 20,000 jobs for new physicians. Because the market has complementarities between applicants and between positions, the theory of simple matching markets does not apply directly. However, computational experiments show the theory provides good approximations. Furthermore, the set of stable matchings, and the opportunities for strategic manipulation, are surprisingly small. A new kind of “core convergence” result explains this; that each applicant interviews only a small fraction of available positions is important. We also describe engineering aspects of the design process. (JEL C78, B41, J44)

Nobel de Economia em 2012

Lloyd Shapley: Teoria de emparelhamento estável e algoritmo G-S.

Alvin Roth: Aplicação do algoritmo G-S no problema de **hospitais e estudantes**, e **doadores de órgãos com pacientes**.



Lloyd Shapley



Alvin Roth



Uma Aplicação Moderna

Content delivery networks (CDNs). Distribute much of world's content on web.

- **User.** Preferences based on latency and packet loss.
- **Web server.** Preferences based on costs of bandwidth and co-location.
- **Goal.** Assign billions of users to servers, every 10 seconds.

Algorithmic Nuggets in Content Delivery

Bruce M. Maggs
Duke and Akamai
bmm@cs.duke.edu

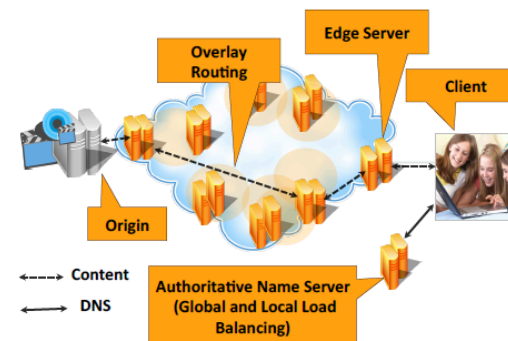
Ramesh K. Sitaraman
UMass, Amherst and Akamai
ramesh@cs.umass.edu

This article is an editorial note submitted to CCR. It has NOT been peer reviewed.

The authors take full responsibility for this article's technical content. Comments can be posted through CCR Online.

ABSTRACT

This paper “peeks under the covers” at the subsystems that provide the basic functionality of a leading content delivery network. Based on our experiences in building one of the largest distributed systems in the world, we illustrate how sophisticated algorithmic research has been adapted to balance the load between and within server clusters, manage the caches on servers, select paths through an overlay routing network, and elect leaders in various contexts. In each instance, we first explain the theory underlying the algorithms, then introduce practical considerations not captured by the theoretical models, and finally describe what is implemented in practice. Through these examples, we highlight the role of algorithmic research in the design of complex networked systems. The paper also illustrates the close synergy that exists between research and industry where research ideas cross over into products and product requirements drive future research.



Implementação em Python

Implementação em Python

Github

Python Code

↺ Start Over

▶ Run Code

```
1 # Prof. Bruno Iochins Grisci
2
3 import random
4 from collections import deque
5 import matplotlib.pyplot as plt
6
7 def plot_curves(i_rand, i_hard, i_easy):
8     # Build aligned x/y arrays
9     xs = sorted(i_rand.keys())
10    y_rand = [i_rand[n] for n in xs]
11    y_hard = [i_hard[n] for n in xs]
12    y_easy = [i_easy[n] for n in xs]
13
14    # (Optional) theoretical references
15    y_linear = xs # ~ n (easy)
16    y_quadratic_half = [n*(n+1)//2 for n in xs] # ~ n^2/2 (hard-ish)
17    y_quadratic = [n*n for n in xs] # ~ n^2 (hard-ish)
18
19    plt.figure(figsize=(8, 5))
20    plt.plot(xs, y_rand, marker='o', label='Random')
21    plt.plot(xs, y_hard, marker='s', label='Hard-ish (identical hospitals)')
22    plt.plot(xs, y_easy, marker='^', label='Easy (unique top choices)')
```

```

23
24 # (Optional) add theory lines
25 plt.plot(xs, y_linear, linestyle='--', label='~ n (theory)')
26 plt.plot(xs, y_quadratic_half, linestyle='--', label='~ n(n+1)/2 (theory)')
27 # plt.plot(xs, y_quadratic, linestyle='--', label='~ n*n (theory)')
28
29 plt.title('Gale-Shapley: iterations vs problem size n')
30 plt.xlabel('n (|H| = |E|)')
31 plt.ylabel('Number of loop iterations')
32 plt.legend()
33 plt.grid(True)
34 plt.tight_layout()
35 #plt.yscale('log')
36 plt.show()
37
38 def random_instance(n, seed=None):
39     """
40     Create a random one-to-one instance with n 'hospitals' and n 'students'.
41     Names: H1..Hn and E1..En.
42     """
43     rng = random.Random(seed)
44     men = [f"M{i}" for i in range(1, n+1)]
45     women = [f"W{i}" for i in range(1, n+1)]
46
47     m_prefs = {m: rng.sample(women, k=n) for m in men}
48     w_prefs = {w: rng.sample(men, k=n) for w in women}
49     return m_prefs, w_prefs
50
51 def worstish_instance(n):
52     """
53     Identical hospitals, reversed students (tends to increase proposals).

```

```

54 """
55 men = [f"M{i}" for i in range(1, n+1)]
56 women = [f"W{i}" for i in range(1, n+1)]
57 # All men rank W1 > W2 > ... > Wn
58 m_prefs = {m: women[:] for m in men}
59 # Each woman ranks men in reverse (Mn > ... > M1)
60 rev_men = men[::-1]
61 w_prefs = {w: rev_men[:] for w in women}
62 return m_prefs, w_prefs
63
64 def easy_instance(n):
65     """
66     Each man's top choice is unique and each student ranks their designated hospital first.
67     """
68     men = [f"M{i}" for i in range(1, n+1)]
69     women = [f"W{i}" for i in range(1, n+1)]
70     # Mi's top choice is Wi; each woman Wi ranks Mi first
71     m_prefs = {
72         men[i]: [women[i]] + [w for w in women if w != women[i]]
73         for i in range(n)
74     }
75     w_prefs = {
76         women[i]: [men[i]] + [m for m in men if m != men[i]]
77         for i in range(n)
78     }
79     return m_prefs, w_prefs
80
81 def gale_shapley(m_prefs: dict[str, list[str]], w_prefs: dict[str, list[str]], verbose=False):
82     """
83     m_prefs: {hospital: [students in descending preference]}
84     w_prefs: {student: [hospitals in descending preference]}

```

```

85     returns: set of (hospital, student) pairs
86     """
87     # Precompute each student's ranking of hospitals for comparisons
88     w_rank = {w: {m: i for i, m in enumerate(pref_list)} for w, pref_list in w_prefs.items()}
89
90     # https://www.geeksforgeeks.org/python/deque-in-python/
91     free_men = deque(m_prefs.keys())          # all hospitals start free
92     next_idx = {m: 0 for m in m_prefs}        # next woman to propose to per man
93     engaged_to = {}                          # woman -> man (current fiancé)
94
95     i_loop = 0
96     while free_men:
97         i_loop += 1
98         m = free_men.popleft()
99
100        # If m has already proposed to every woman, skip (no match possible)
101        if next_idx[m] >= len(m_prefs[m]):
102            if verbose: print(f'{m} has already proposed to everyone.')
103            continue
104
105        # Highest-ranked woman m hasn't proposed to yet
106        w = m_prefs[m][next_idx[m]]
107        next_idx[m] += 1
108
109        if verbose: print(f'{m} proposes to {w}.')
110
111        if w not in engaged_to:                # w is free
112            engaged_to[w] = m                  # (m, w) become engaged
113            if verbose: print(f'{w} is free, {m} and {w} become engaged.')
114        else:
115            m_prime = engaged_to[w]            # w is engaged to m'

```

```

116         # If w prefers current fiancé m' over m
117         if w_rank[w][m_prime] < w_rank[w][m]:
118             # m remains free; try next woman later
119             free_men.append(m)
120             if verbose: print(f'{m} remains free.')
121         else:
122             # w prefers m to m'
123             engaged_to[w] = m          # (m, w) become engaged
124             free_men.append(m_prime)    # m' becomes free
125             if verbose: print(f'{m} and {w} become engaged, {m_prime} becomes free.')
126
127     # If m still hasn't proposed to everyone and is free, he'll re-enter loop via queue
128     # Loop continues while there exists a free man who hasn't proposed to all women
129
130     if verbose: print(engaged_to)
131
132     # Build result set S of engaged pairs
133     return {(m, w) for w, m in engaged_to.items()}, i_loop
134
135 def main():
136
137     # Demo: https://www.cs.princeton.edu/~wayne/kleinberg-tardos/pdf/01DemoGaleShapley.pdf
138     # hospitals' preference lists
139     HPL = {
140         #1st      2nd      3rd      4th      5th
141         'Atlanta': ['Wayne', 'Val', 'Yolanda', 'Zeus', 'Xavier'],
142         'Boston': ['Yolanda', 'Wayne', 'Val', 'Xavier', 'Zeus'],
143         'Chicago': ['Wayne', 'Zeus', 'Xavier', 'Yolanda', 'Val'],
144         'Detroit': ['Val', 'Yolanda', 'Xavier', 'Wayne', 'Zeus'],
145         'El Paso': ['Wayne', 'Yolanda', 'Val', 'Zeus', 'Xavier'],
146     }

```

```
147 # students' preference lists
148 SPL = {
149     #1st      2nd      3rd      4th      5th
150     'Val':    ['El Paso', 'Atlanta', 'Boston', 'Detroit', 'Chicago'],
151     'Wayne':  ['Chicago', 'Boston', 'Detroit', 'Atlanta', 'El Paso'],
152     'Xavier': ['Boston', 'Chicago', 'Detroit', 'El Paso', 'Atlanta'],
153     'Yolanda': ['Atlanta', 'El Paso', 'Detroit', 'Chicago', 'Boston'],
154     'Zeus':   ['Detroit', 'Boston', 'El Paso', 'Chicago', 'Atlanta'],
155 }
156
157 # Pior caso dos slides de aula
158 HOSPITAIS = {
159     'V': ['A', 'B', 'C', 'D', 'E'],
160     'W': ['B', 'C', 'D', 'A', 'E'],
161     'X': ['C', 'D', 'A', 'B', 'E'],
162     'Y': ['D', 'A', 'B', 'C', 'E'],
163     'Z': ['A', 'B', 'C', 'D', 'E'],
164 }
165
166 ESTUDANTES = {
167     'A': ['W', 'X', 'Y', 'Z', 'V'],
168     'B': ['X', 'Y', 'Z', 'V', 'W'],
169     'C': ['Y', 'Z', 'V', 'W', 'X'],
170     'D': ['Z', 'V', 'W', 'X', 'Y'],
171     'E': ['V', 'W', 'X', 'Y', 'Z'],
172 }
173
174 # Demo:
175 print(gale_shapley(HPL, SPL, True))
176
177 # Worst case:
178 print(gale_shapley(HOSPITAIS, ESTUDANTES, True))
```

```
178
179 # Analysis:
180 v i_rand = {}
181 i_hard = {}
182 i_easy = {}
183
184 v for n in range(10, 1000, 100):
185     mp, wm = random_instance(n)
186     i_rand[n] = gale_shapley(mp, wm)[1]
187
188 v for n in range(10, 1000, 100):
189     mp, wm = worstish_instance(n)
190     i_hard[n] = gale_shapley(mp, wm)[1]
191
192 for n in range(10, 1000, 100):
193     mp, wm = easy_instance(n)
194     i_easy[n] = gale_shapley(mp, wm)[1]
195
196 print('RANDOM')
197 print(i_rand)
198 print('WORST')
199 print(i_hard)
200 print('EASY')
201 print(i_easy)
202
```